Multi-Source Candidate Data Transformer - Technical Design

Problem Frame

The transformer ingests candidate data from multiple sources and emits one canonical candidate profile. This implementation handles one structured source, a recruiter CSV, and one unstructured source, recruiter notes. The goal is deterministic, explainable merging rather than a broad but brittle parser.

Pipeline

The pipeline is:

```
Load sources -> Extract fields -> Normalize values -> Match candidate -> Merge conflicts -> Add confidence/provenance -> Apply output config -> Validate JSON
```

The CSV reader extracts name, email, phone, current company, and title. The notes reader uses simple labels and regex extraction for headline, location, email, phone, years of experience, and skill mentions.

Canonical Schema And Normalization

The internal profile contains candidate_id, full_name, emails, phones, location, links, headline, years_experience, skills, experience, education, provenance, overall_confidence, and warnings. Emails are trimmed and lowercased. Phones are normalized to a simple E.164-style format. Locations are split into city, region, and ISO-like country code. Skills are mapped through a small alias table, for example py to Python and js to JavaScript.

Merge And Confidence Policy

Email is the strongest match key, followed by phone. CSV wins for identity and contact fields because it is structured and recruiter-entered. Notes enrich the record with headline, location, years of experience, and skills. Conflicts do not crash the run; the pipeline keeps the earlier higher-priority value, records a warning, and lowers overall confidence. Provenance entries store field, source, method, and confidence so every chosen value is traceable.

Runtime Config Layer

The engine always builds the full canonical profile first. A separate projection layer accepts a runtime JSON config that can select fields, rename fields with `from`, project array paths like `skills[].name`, toggle confidence/provenance, and choose missing behavior: `null`, `omit`, or `error`. This keeps output customization separate from source parsing and merging.

Edge Cases And Scope

Handled edge cases include missing CSV, empty notes, malformed rows, invalid emails, conflicting phones, and unknown skill spelling. Unknown values become null, omitted, or errors depending on config. Under time pressure, I deliberately leave out LinkedIn/GitHub APIs, resume parsing, fuzzy multi-candidate matching, and ML-based entity extraction so the submitted core stays reliable and explainable.